Docente: Noslin A. Guerra Cabeza
INACAP, Sede San Pedro de la Paz

Informe Técnico — ES2

Implementación y Persistencia Segura · Caso EcoTech Solutions

Unidad de Aprendizaje 2:
Implementación de Software Orientado a Objetos en Python.

Aprendizaje esperado
2.1. Construye una solución de software en Python para resolver una problemática planteada, considerando el paradigma de programación orientada a objetos, la conexión segura a bases de datos y la aplicación de criterios técnicos para validar e integrar recomendaciones generadas por herramientas de IA.

Actividades
Desarrollar la aplicación CRUD en Python a partir del modelo UML validado, integrar la persistencia relacional con consultas parametrizadas, implementar manejo de excepciones y transacciones atómicas, auditar el código generado por IA y documentar la solución para la defensa argumentativa individual.

Evaluación
- Evaluación Sumativa N°2 (ES2)
- Ponderación: 30% de la asignatura
- Estructura: 50% Producto grupal + 50% Defensa individual
- Modalidad: Grupal (2 a 3 estudiantes) con Líder de Proyecto rotativo
- Instrumento: Escala de Apreciación oficial (AAI - 20 indicadores)
- Entrega: Archivo ZIP con código + Informe en Word/PDF en el AAI

1. Presentación

Este informe documenta la entrega de la Etapa II del proyecto "Sistema de Gestión de Empleados" de EcoTech Solutions: la arquitectura modular del código fuente en Python, la implementación rigurosa de las clases del dominio y sus pilares de POO, la capa de persistencia relacional con consultas 100% parametrizadas, el control de errores con excepciones de negocio y transacciones atómicas (commit/rollback), la bitácora de auditoría crítica de IA y la trazabilidad de aportes de cada integrante para la defensa.

Criterios de evaluación

2.1.1.- Implementa la estructura de clases y métodos en Python a partir de un diagrama UML validado.
2.1.2.- Aplica principios de programación orientada a objetos en la solución desarrollada en Python, con coherencia con el modelo diseñado y sin automatización del código.
2.1.3.- Emplea librerías oficiales de Python para la conexión a bases de datos, según el sistema gestor seleccionado y los requerimientos de la solución.
2.1.4.- Utiliza mecanismos de manejo de errores y excepciones mediante estructuras de control orientadas a la estabilidad y seguridad del software.
2.1.5.- Aplica criterios técnicos en la validación del código generado por herramientas de inteligencia artificial, justificando su adopción, modificación o descarte.

Equipo de Trabajo y Líder de Proyecto

Líder de Proyecto: Amanda Hidalgo · 21694333-3

Integrante 2: Roberto Gajardo · 21582058-0

Integrante 3: Jesús Capetillo · 22711930-6

Sección: TI3021

Docente: Noslin A. Guerra Cabeza
INACAP, Sede San Pedro de la Paz

Índice de Contenidos

1. Introducción y Alcance de la Solución 3
2. Arquitectura y Organización del Código Fuente 4
2.1 Árbol de Directorios del Proyecto 4
2.2 Justificación de la Separación de Capas 5
2.3 Requisitos del Entorno y Guía de Ejecución 5
3. Implementación del Modelo Orientado a Objetos en Python 6
3.1 Correspondencia entre Diagrama UML y Clases en Python 6
3.2 Clases de Dominio y Constructores 7
3.3 Aplicación de Principios de POO 8
4. Capa de Persistencia y Acceso Seguro a Datos 9
4.1 Conexión Oficial y Protección de Credenciales 9
4.2 Esquema Relacional de Base de Datos 10
4.3 Operaciones CRUD Parametrizadas en el Repositorio 11
4.4 Función de Mapeo Centralizado (Tupla ➔ Objeto) 11
4.5 Prueba de Blindaje contra Inyección SQL 12
5. Manejo de Errores, Excepciones y Transacciones Atómicas 14
5.1 Jerarquía de Excepciones Propias del Dominio 14
5.2 Bloques try - except Específicos y Validación de Entradas 15
5.3 Operación Transaccional Atómica (Commit y Rollback) 16
6. Auditoría y Validación Crítica de Código Generado por IA 17
6.1 Registro de Herramientas e Interacciones con IA 17
6.2 Matriz de Auditoría y Decisiones Técnicas 17
7. Trazabilidad de Aportes y Preparación de la Defensa Individual 18
7.1 Matriz de Contribución y Autoría del Código 18
7.2 Registro de Gestión del Líder de Proyecto 18
7.3 Cuestionario de Preparación para la Defensa Individual 19
8. Referencias Bibliográficas 19

Docente: Noslin A. Guerra Cabeza
INACAP, Sede San Pedro de la Paz

1. Introducción y Alcance de la Solución

> EcoTech Solutions administra su nómina, sus proyectos y el control de horas en planillas independientes, sin un criterio único de acceso ni de seguridad. En la Etapa I el equipo modeló esa realidad con diagramas de clases UML; en esta Etapa II, ese modelo validado se convirtió en una aplicación funcional en Python, conectada a una base de datos relacional y con control de errores, de modo que la información se gestione de forma segura y consistente.
>
> El objetivo de la aplicación es gestionar **por consola** a los empleados y departamentos de la empresa: registrar, consultar, actualizar y eliminar, además de registrar un empleado junto con su credencial de acceso en una operación atómica. Las entidades **Empleado, Departamento y Credencial** se persisten en SQLite. Las demás clases del modelo (**Proyecto, ParteDiario, ParticipacionProyecto, Perfil, Permiso, Reporte**) están implementadas según el UML y se ejercitan en la demostración automática, pero no se guardan en la base de datos. La aplicación no incluye interfaz gráfica porque está fuera del alcance de la Unidad 2 y no es evaluada.

Docente: Noslin A. Guerra Cabeza
INACAP, Sede San Pedro de la Paz

2. Arquitectura y Organización del Código Fuente

2.1 Árbol de Directorios del Proyecto

```
ecotech_solutions/
│
├── dominio/                        # Clases del modelo UML (POO pura, sin SQL)
│   ├── __init__.py
│   ├── persona.py                  # «abstract» Persona: datos de contacto comunes
│   ├── empleado.py                 # Empleado ES UNA Persona
│   ├── departamento.py             # Agrupa empleados (agregación)
│   ├── proyecto.py                 # Proyecto: horas totales
│   ├── participacion_proyecto.py   # Clase de asociación Empleado–Proyecto
│   ├── parte_diario.py             # ParteDiario con esJornadaValida()
│   ├── credencial.py               # Credencial: clave cifrada privada
│   ├── perfil.py / permiso.py      # Perfil contiene Permisos (composición)
│   ├── gestor_cifrado.py           # Hash PBKDF2 con sal (hashlib)
│   ├── reporte.py                  # «abstract» Reporte
│   ├── reporte_pdf.py / reporte_excel.py   # exportar() polimórfico
│   ├── validaciones.py             # Validaciones compartidas
│   └── excepciones.py              # Excepciones propias del dominio
│
├── repositorio/                    # Capa de persistencia (todo el SQL)
│   ├── __init__.py
│   ├── conexion.py                 # Conexión sqlite3 y variables de entorno
│   ├── base_repo.py                # Ejecución común y errores
│   ├── empleado_repo.py            # CRUD de Empleado y transacción
│   └── departamento_repo.py        # CRUD de Departamento
│
├── datos/
│   └── schema.sql                  # Tablas con llaves primarias y foráneas
│
├── .env.example                    # Plantilla de variables de entorno
├── .gitignore                      # Excluye .env, __pycache__ y .db
├── requirements.txt                # Sin dependencias externas
├── main.py                         # Menú por consola y demostración (--demo)
└── README.md                       # Ejecución rápida
```

2.2 Justificación de la Separación de Capas

> Las clases del dominio (Persona, Empleado, Departamento…) representan conceptos del negocio y sus reglas, por ejemplo calcular la antigüedad o validar un sueldo. Por eso **no contienen sentencias SQL**: todo `INSERT`, `SELECT`, `UPDATE` y `DELETE` vive exclusivamente en `repositorio/`. Cada clase tiene así una sola razón para cambiar (principio de responsabilidad única, SRP): el dominio cambia si cambian las reglas de la empresa; el repositorio cambia si cambia la forma de guardar los datos.
>
> **Beneficio ante un cambio de SQLite a PostgreSQL o Supabase:** solo se modifica la carpeta `repositorio/` (la librería de conexión, como `psycopg2`, y el marcador de parámetros de `?` a `%s`). Las clases del dominio, el menú y las reglas de negocio quedan intactos, porque nunca supieron que existía una base de datos. Además, el dominio puede probarse sin base de datos, como se hace en la demostración con los reportes y los partes diarios.

2.3 Requisitos del Entorno y Guía de Ejecución

Paso	Comando o Acción	Resultado Esperado
1. Entorno virtual	`python -m venv venv` `venv\Scripts\activate` (Windows) o `source venv/bin/activate`	Entorno virtual aislado creado y activo
2. Dependencias	`pip install -r requirements.txt`	El proyecto usa solo la biblioteca estándar de Python 3.10+; no hay paquetes que instalar
3. Configuración	Copiar `.env.example` a `.env` (opcional: define ECOTECH_DB_PATH)	Ruta de la base disponible por variable de entorno; si no existe, se usa `ecotech.db`
4. Inicialización	`python -c "from repositorio.conexion import inicializar_bd; inicializar_bd()"`	Tablas relacionales creadas con llaves foráneas (`main.py` también las crea al iniciar)
5. Ejecución	`python main.py` (menú) o `python main.py --demo` (demostración automática)	Mensaje de conexión exitosa y sistema en consola demostrando las operaciones

Docente: Noslin A. Guerra Cabeza
INACAP, Sede San Pedro de la Paz

3. Implementación del Modelo Orientado a Objetos en Python

3.1 Correspondencia entre Diagrama UML y Clases en Python

Mínimo exigido: Al menos 3 clases del modelo mapeadas con su archivo, constructor y atributos.

Entidad en Diagrama UML	Clase en Python	Archivo Fuente	Responsabilidad Principal
Persona «abstract»	Persona	dominio/persona.py	Entidad base abstracta con datos de contacto comunes. Constructor `__init__(idPersona, nombreCompleto, direccionParticular, telefonoContacto, correoElectronico)`; `__idPersona` privado y el resto protegidos.
Empleado	Empleado	dominio/empleado.py	Hereda de Persona. Constructor agrega `sueldoBase` (privado) y `fechaIngreso`; calcula `antiguedadEnAnios()` y cambia de departamento.
Departamento	Departamento	dominio/departamento.py	Agrupa empleados: `incorporarEmpleado()`, `retirarEmpleado()`, `nomina()`. Constructor `__init__(codigoDepto, nombreDepto)`.
Proyecto	Proyecto	dominio/proyecto.py	Recibe participaciones y partes diarios; `totalHorasInvertidas()`. Constructor `__init__(codigoProyecto, tituloProyecto, resumen, fechaLanzamiento)`.
ParticipacionProyecto	ParticipacionProyecto	dominio/participacion_proyecto.py	Clase de asociación Empleado–Proyecto con `fechaIncorporacion` y `__enCurso`; `cerrarParticipacion()`.
ParteDiario	ParteDiario	dominio/parte_diario.py	Registra una jornada; `esJornadaValida()` rechaza horas fuera de 0 a 24. Constructor `__init__(folio, empleado, proyecto, jornada, horasImputadas, detalleActividad)`.
Credencial	Credencial	dominio/credencial.py	Acceso del empleado (0..1); `__claveCifrada` y `__habilitada` privados; `validarAcceso()` y `actualizarClave()`.
Perfil / Permiso	Perfil, Permiso	dominio/perfil.py, dominio/permiso.py	El perfil agrupa permisos (composición) con `incluirPermiso()`.
GestorCifrado	GestorCifrado	dominio/gestor_cifrado.py	Cifra y valida claves con hash PBKDF2-SHA256 y sal (`encriptar()`, `validar()`).
Reporte «abstract»	Reporte	dominio/reporte.py	Construye el contenido (`construir()`) y obliga a las subclases a implementar `exportar()`.
ReportePDF / ReporteExcel	ReportePDF, ReporteExcel	dominio/reporte_pdf.py, dominio/reporte_excel.py	Heredan de Reporte y redefinen `exportar()` (polimorfismo).

3.2 Clases de Dominio y Constructores

```
# dominio/persona.py
class Persona(ABC):
    """«abstract» Persona"""

    def __init__(self, idPersona, nombreCompleto, direccionParticular, telefonoContacto,
correoElectronico):
        self.__idPersona = entero_positivo(idPersona, "El id")  # -idPersona
        self._nombreCompleto = texto_no_vacio(nombreCompleto, "El nombre")  # #nombreCompleto
        self._direccionParticular = direccionParticular or ""  # #direccionParticular
        self._telefonoContacto = telefonoContacto or ""  # #telefonoContacto
        self._correoElectronico = correo_valido(correoElectronico)  # #correoElectronico

# dominio/empleado.py
class Empleado(Persona):
    """Empleado ES UNA Persona. Pertenece a un Departamento (agregación)."""

    def __init__(self, idPersona, nombreCompleto, direccionParticular, telefonoContacto,
                 correoElectronico, sueldoBase, fechaIngreso):
        super().__init__(idPersona, nombreCompleto, direccionParticular, telefonoContacto,
correoElectronico)
        self.__sueldoBase = numero_positivo(sueldoBase, "El sueldo base")  # -sueldoBase : float
        self.fechaIngreso = fecha_valida(fechaIngreso, "La fecha de ingreso")  # +fechaIngreso :
date
        self.__departamento = None
        self.__credencial = None  # 0..1 Credencial
    ...
    def antiguedadEnAnios(self):
        hoy = date.today()
        anios = hoy.year - self.fechaIngreso.year
        if (hoy.month, hoy.day) < (self.fechaIngreso.month, self.fechaIngreso.day):
            anios -= 1
        return anios
```

Persona es una clase abstracta (ABC): no se puede instanciar y obliga a sus subclases a implementar `descripcion()`. Su constructor guarda el id como atributo **privado** (`__idPersona`) y el nombre, dirección, teléfono y correo como atributos **protegidos** (`_nombreCompleto`, etc.), tal como lo indica el UML (`-` y `#`). Cada dato pasa por una función de validación antes de asignarse.

Empleado hereda de Persona y llama a `super().__init__(...)` para reutilizar ese constructor; luego agrega sus atributos propios, `__sueldoBase` (privado, validado) y `fechaIngreso`. El método de negocio `antiguedadEnAnios()` calcula los años completos desde la fecha de ingreso. Ninguna de estas clases contiene código de base de datos.

Docente: Noslin A. Guerra Cabeza
INACAP, Sede San Pedro de la Paz

3.3 Aplicación de Principios de POO

A) Encapsulamiento y Validación de Datos

Usamos tres técnicas: (1) atributos **privados** (`__sueldoBase`, `__claveCifrada`, `__idPersona`) y **protegidos** (`_nombreCompleto`, `_correoElectronico`); (2) `@property` para leer y *setters* que validan antes de asignar; (3) atributos de **solo lectura** (`claveCifrada`, `idPersona`), que no tienen setter y por eso no se pueden reemplazar desde fuera.

```
    @property
    def sueldoBase(self):
        return self.__sueldoBase

    @sueldoBase.setter
    def sueldoBase(self, valor):
        self.__sueldoBase = numero_positivo(valor, "El sueldo base")

# dominio/validaciones.py
def numero_positivo(valor, campo):
    if isinstance(valor, bool) or not isinstance(valor, (int, float)) or valor <= 0:
        raise DatoInvalidoError(f"{campo} debe ser un número mayor a 0.")
    return float(valor)
```

Error que previene: que se asigne un sueldo negativo, cero o un texto; en ese caso se lanza `DatoInvalidoError` antes de que el dato llegue a la base. La clave nunca se guarda en texto plano: `Credencial` la transforma con `GestorCifrado.encriptar()` y solo conserva el hash.

B) Herencia y Reutilización de Código

Dónde: `Empleado(Persona)` en `dominio/empleado.py` y `ReportePDF(Reporte)` / `ReporteExcel(Reporte)`. `super().__init__(...)` reutiliza el inicializador de `Persona`, de modo que id, nombre, dirección, teléfono y correo (con sus validaciones) se escriben una sola vez.

Por qué herencia: Empleado *es una* Persona y comparte todos sus datos de contacto; los reportes comparten `construir()` y solo difieren en el formato de `exportar()`. Además se evita duplicar código con `validaciones.py` (funciones reutilizadas por todas las clases) y con `BaseRepositorio` (ejecución y manejo de errores reutilizados por los repositorios).

Decisión de diseño de la Unidad 1: no se crearon subclases para roles temporales como «Gerente» o «Administrador», porque son cargos que una persona ejerce dentro de un departamento o con una credencial, no tipos distintos de objeto. El código respeta esa decisión: no existe ninguna clase `Gerente`.

C) Polimorfismo

```
# main.py, demostración (paso 7)
for reporte in (ReportePDF("horas por proyecto"), ReporteExcel("horas por proyecto")):
    texto = reporte.construir([marcela.departamento], [proyecto])
    print(f"  {type(reporte).__name__}.exportar() -> {len(reporte.exportar(texto))} bytes")
```

La colección mixta `(ReportePDF, ReporteExcel)` se recorre llamando al mismo método `construir()` y `exportar()` sin ningún `if type(x) == ...` ni `isinstance(...)`: cada objeto sabe cómo exportarse. Para agregar un formato nuevo (por ejemplo CSV) basta crear otra subclase de `Reporte`, sin modificar el código existente (principio abierto/cerrado). Salida real: `ReportePDF.exportar() -> 184 bytes`, `ReporteExcel.exportar() -> 164 bytes`.

Docente: Noslin A. Guerra Cabeza
INACAP, Sede San Pedro de la Paz

4. Capa de Persistencia y Acceso Seguro a Datos

4.1 Conexión Oficial y Protección de Credenciales

Parámetro / Configuración	Valor en Entorno Local	Mecanismo de Protección
SGBD seleccionado	SQLite 3	Motor relacional estándar, portable y sin servidor que instalar
Librería oficial de Python	`sqlite3` (biblioteca estándar)	Driver oficial incluido en Python, con soporte de consultas parametrizadas
Archivo de variables	`.env` (variable ECOTECH_DB_PATH)	Registrado en `.gitignore`; nunca se sube al repositorio
Archivo de ejemplo versionado	`.env.example`	Plantilla sin credenciales reales, sí se sube a Git

ConexionBD abre la base con `sqlite3.connect(ruta)` y activa las llaves foráneas con `PRAGMA foreign_keys = ON` en cada conexión. La ruta se lee desde la variable de entorno ECOTECH_DB_PATH: la función `cargar_env()` toma las líneas CLAVE=VALOR del archivo `.env` (si existe) y una variable ya definida en el sistema tiene prioridad. Como SQLite no usa usuario ni contraseña, el `.env` solo contiene la ruta; igual queda excluido de Git por buena práctica, y se entrega `.env.example` como plantilla.

```
# repositorio/conexion.py
    def conectar(self):
        try:
            self.__conexion = sqlite3.connect(self.__ruta)
            self.__conexion.execute("PRAGMA foreign_keys = ON")  # activa las llaves foráneas
        except sqlite3.Error as error:
            self.__conexion = None
            raise ErrorConexionBD(f"No fue posible conectar con la base de datos: {error}") from
error
        return self.__conexion
```

Si la conexión falla, se captura `sqlite3.Error` y se lanza `ErrorConexionBD` con un mensaje claro; `main.py` lo muestra y termina de forma ordenada. Evidencia de la conexión exitosa (salida real del programa):

```
>>> 1. Conexión a la base de datos
Conexión exitosa a la base de datos (SQLite 3.50.4).
```

4.2 Esquema Relacional de Base de Datos

```
-- datos/schema.sql
CREATE TABLE IF NOT EXISTS departamento (
    codigo_depto INTEGER PRIMARY KEY,
    nombre_depto TEXT NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS empleado (
    id_persona     INTEGER PRIMARY KEY,
    nombre_completo TEXT NOT NULL,
    direccion      TEXT,
    telefono       TEXT,
    correo         TEXT NOT NULL UNIQUE,
    sueldo_base    REAL NOT NULL CHECK (sueldo_base > 0),
    fecha_ingreso  TEXT NOT NULL,
    codigo_depto   INTEGER,
    activo         INTEGER NOT NULL DEFAULT 1 CHECK (activo IN (0, 1)),
    FOREIGN KEY (codigo_depto) REFERENCES departamento (codigo_depto) ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS credencial (
    nombre_usuario TEXT PRIMARY KEY,
    clave_cifrada  TEXT NOT NULL,
    habilitada     INTEGER NOT NULL DEFAULT 1 CHECK (habilitada IN (0, 1)),
    perfil         TEXT NOT NULL,
    id_persona     INTEGER NOT NULL UNIQUE,
    FOREIGN KEY (id_persona) REFERENCES empleado (id_persona) ON DELETE CASCADE
);
```

Cada tabla define su llave primaria (`PRIMARY KEY`); `empleado` referencia a `departamento` y `credencial` referencia a `empleado` mediante `FOREIGN KEY`. Las restricciones `UNIQUE` (correo, nombre de departamento, usuario) y `CHECK` (sueldo mayor a 0) son una tercera defensa si algún dato se saltara las validaciones del programa. La integridad referencial se activa con `PRAGMA foreign_keys = ON;`, que `ConexionBD` ejecuta en cada conexión (SQLite la ignora si no se activa). Con `ON DELETE SET NULL`, al eliminar un departamento sus empleados quedan sin departamento; con `ON DELETE CASCADE`, la credencial se elimina junto con su empleado.

4.3 Operaciones CRUD Parametrizadas en el Repositorio

Mínimo exigido: CRUD completo para al menos dos entidades del sistema.

Operación CRUD	Método en Repositorio	Sentencia SQL Parametrizada	Explicación Breve
Crear (Create)	`guardar(empleado)`	`INSERT INTO empleado (id_persona, nombre_completo, direccion, telefono, correo, sueldo_base, fecha_ingreso, codigo_depto) VALUES (?, ?, ?, ?, ?, ?, ?, ?)`	Inserta el registro con los valores en una tupla, separados del texto SQL.
Leer uno (Read)	`obtener_por_id(id_persona)`	`SELECT ... FROM empleado e LEFT JOIN departamento d ... WHERE e.id_persona = ? AND e.activo = 1`	Consulta por llave primaria con parámetro aislado.
Listar todos (Read)	`listar_activos()`	`SELECT ... WHERE e.activo = 1 ORDER BY e.id_persona`	Recupera las filas y usa el mapper para devolver objetos Empleado.
Buscar (Read)	`buscar_por_nombre(nombre)`	`SELECT ... WHERE e.nombre_completo = ? AND e.activo = 1`	Búsqueda con texto del usuario, 100% parametrizada.
Actualizar (Update)	`actualizar(empleado)`	`UPDATE empleado SET nombre_completo = ?, direccion = ?, telefono = ?, correo = ?, sueldo_base = ?, codigo_depto = ? WHERE id_persona = ? AND activo = 1`	Modifica datos existentes sin concatenar variables.
Eliminar (Delete)	`eliminar(id_persona)`	`UPDATE empleado SET activo = 0 WHERE id_persona = ? AND activo = 1`	Borrado lógico: el empleado se conserva pero deja de aparecer en los listados.

> El CRUD de **Departamento** (`DepartamentoRepositorio`) es la segunda entidad con operaciones completas; en total se cubre el mínimo exigido de dos entidades. Todos los métodos de actualizar y eliminar verifican `cursor.rowcount` y lanzan `RegistroNoEncontradoError` si el registro no existe.

```
# repositorio/departamento_repo.py  (segunda entidad con CRUD completo)
guardar(depto)            INSERT INTO departamento (codigo_depto, nombre_depto) VALUES (?, ?)
obtener_por_codigo(cod)   SELECT codigo_depto, nombre_depto FROM departamento WHERE codigo_depto = ?
listar()                  SELECT codigo_depto, nombre_depto FROM departamento ORDER BY codigo_depto
actualizar(depto)         UPDATE departamento SET nombre_depto = ? WHERE codigo_depto = ?
eliminar(codigo)          DELETE FROM departamento WHERE codigo_depto = ?   (borrado físico)
```

> Resultado real del CRUD (`python main.py --demo`, paso 2):

```
>>> 2. CRUD de Departamento y Empleado
Listado:
  [1] Marcela Iturra <marcela@ecotech.cl> - Sustentabilidad Industrial
  [2] Luis Pérez <luis@ecotech.cl> - sin departamento
Búsqueda por id 1: [1] Marcela Iturra <marcela@ecotech.cl> - Sustentabilidad Industrial
Sueldo actualizado: 1650000.0
Activos tras eliminar el id 2: ['Marcela Iturra']
Departamentos: ['Sustentabilidad Industrial']
```

4.4 Función de Mapeo Centralizado (Tupla ➔ Objeto)

```
# repositorio/empleado_repo.py
    @staticmethod
```

```
    def _fila_a_empleado(fila, deptos=None):
        """Único punto donde una fila (tupla) de la BD se transforma en un objeto Empleado."""
        empleado = Empleado(fila[0], fila[1], fila[2], fila[3], fila[4], fila[5],
                            date.fromisoformat(fila[6]))
        if fila[7] is not None:
            deptos = deptos if deptos is not None else {}
            depto = deptos.setdefault(fila[7], Departamento(fila[7], fila[8]))
            empleado.cambiarDepartamento(depto)
        return empleado
```

`_fila_a_empleado()` es el **único** punto donde una fila (tupla) devuelta por SQLite se transforma en un objeto `Empleado` (convierte la fecha de texto a `date` y reconstruye la relación con su `Departamento`). Todos los métodos de lectura (`obtener_por_id`, `buscar_por_nombre`, `listar_activos`) lo reutilizan; si cambia una columna de la tabla, se corrige en un solo lugar y no pueden aparecer inconsistencias entre distintas consultas.

4.5 Prueba de Blindaje contra Inyección SQL

```
# repositorio/empleado_repo.py
    def buscar_por_nombre(self, nombre):
        """Búsqueda exacta por nombre. El valor viaja como parámetro, nunca dentro del texto SQL."""
        filas = self._consultar(_BASE_SELECT + "WHERE e.nombre_completo = ? AND e.activo = 1",
(nombre,))
        deptos = {}
        return [self._fila_a_empleado(f, deptos) for f in filas]
```

Prueba ejecutada con el payload `' OR '1'='1` como argumento (salida real del programa, `python main.py --demo`, paso 4):

```
>>> 4. Prueba de inyección SQL con ' OR '1'='1
buscar_por_nombre("' OR '1'='1") devolvió 0 resultado(s): la consulta parametrizada trató el payload
como un texto cualquiera.
Búsqueda legítima 'Marcela Iturra': 1 resultado(s)
```

Resultado: la búsqueda devolvió **0 resultados**; el payload fue tratado como un nombre cualquiera y como ningún empleado se llama así, no se filtró ningún dato. La búsqueda legítima por `Marcela Iturra` sí devolvió 1 resultado.

Por qué: en una consulta parametrizada el texto SQL y los valores viajan por separado; el motor compila primero la sentencia con el marcador `?` y después inserta el valor solo como dato, nunca como código. Si la consulta se hubiese armado con un f-string, el `WHERE` habría quedado `nombre_completo = '' OR '1'='1'`, una condición siempre verdadera que devolvería a todos los empleados.

Verificación en el código: una búsqueda de texto sobre `repositorio/` de `execute(f`, `execute(...+`, `execute(...%` y `.format(` no encuentra ninguna coincidencia: ninguna consulta se construye con f-strings, concatenación ni formato de cadenas.

Docente: Noslin A. Guerra Cabeza
INACAP, Sede San Pedro de la Paz

Evidencia de ejecución: salida completa de python main.py --demo en el equipo del grupo (VS Code, SQLite 3.50.4).

Problems Output Debug Console Terminal Ports

```
PS C:\Users\amand\Desktop\INACAP2°S\PROGRAMACIÓN\EV3 GRUPAL\ecotech_solutions_ES2> cd ecotech_solutions
PS C:\Users\amand\Desktop\INACAP2°S\PROGRAMACIÓN\EV3 GRUPAL\ecotech_solutions_ES2\ecotech_solutions> python main.py --demo

>>> 1. Conexión a la base de datos
Conexión exitosa a la base de datos (SQLite 3.50.4).

>>> 2. CRUD de Departamento y Empleado
Listado:
  [1] Marcela Iturra <marcela@ecotech.cl> - Sustentabilidad Industrial
  [2] Luis Pérez <luis@ecotech.cl> - sin departamento
Búsqueda por id 1: [1] Marcela Iturra <marcela@ecotech.cl> - Sustentabilidad Industrial
Sueldo actualizado: 1650000.0
Activos tras eliminar el id 2: ['Marcela Iturra']
Departamentos: ['Sustentabilidad Industrial']

>>> 3. Validaciones y manejo de errores (el programa no se interrumpe)
  Sueldo negativo: DatoInvalidoError -> El sueldo base debe ser un número mayor a 0.
  Correo inválido: DatoInvalidoError -> El correo no tiene un formato válido (ej: nombre@dominio.cl).
  Id repetido: RegistroDuplicadoError -> Ya existe un registro con esos datos únicos.
  Empleado inexistente: RegistroNoEncontradoError -> No existe un empleado activo con id 999.

>>> 4. Prueba de inyección SQL con ' OR '1'='1
buscar_por_nombre("' OR '1'='1") devolvió 0 resultado(s): la consulta parametrizada trató el payload como un texto cualquiera.
Búsqueda legítima 'Marcela Iturra': 1 resultado(s)

>>> 5. Transacción atómica (empleado + credencial)
Caso éxito: empleado 10 guardado -> Jesús Capetillo
Caso fallo: la 2ª escritura falló (Ya existe un registro con esos datos únicos.)
Rollback comprobado: el empleado 11 NO quedó guardado (la BD sigue consistente).

>>> 6. Autenticación con la credencial guardada (la clave se almacena como hash)
Hash guardado: ccea9ce79c8ffc8903f1d509be4c7d...
Clave correcta: True | incorrecta: False

>>> 7. Polimorfismo: una colección mixta de reportes
Parte diario inválido rechazado: Las horas imputadas deben estar entre 0 y 24.
  ReportePDF.exportar() -> 184 bytes
  ReporteExcel.exportar() -> 164 bytes
PS C:\Users\amand\Desktop\INACAP2°S\PROGRAMACIÓN\EV3 GRUPAL\ecotech_solutions_ES2\ecotech_solutions>
```

Docente: Noslin A. Guerra Cabeza
INACAP, Sede San Pedro de la Paz

5. Manejo de Errores, Excepciones y Transacciones Atómicas

5.1 Jerarquía de Excepciones Propias del Dominio

```
# dominio/excepciones.py
class EcoTechError(Exception):
    """Excepción base de todos los errores propios del sistema."""


class DatoInvalidoError(EcoTechError, ValueError):
    """Un dato no cumple las reglas del negocio (vacío, negativo, formato incorrecto)."""


class RegistroDuplicadoError(EcoTechError):
    """Se intentó guardar un registro que ya existe (id, correo o usuario repetido)."""


class RegistroNoEncontradoError(EcoTechError):
    """No existe el registro buscado."""


class ErrorBaseDatos(EcoTechError):
    """Falla de la base de datos que no corresponde a una regla del negocio."""


class ErrorConexionBD(ErrorBaseDatos):
    """No fue posible abrir la conexión con la base de datos."""
```

```
# repositorio/base_repo.py
    @staticmethod
    def traducir_error(error):
        """Convierte una excepción cruda de sqlite3 en una excepción del negocio."""
        if isinstance(error, sqlite3.IntegrityError):
            if "UNIQUE" in str(error) or "PRIMARY KEY" in str(error):
                return RegistroDuplicadoError("Ya existe un registro con esos datos únicos.")
            return ErrorBaseDatos(f"Se violó una regla de integridad: {error}")
        return ErrorBaseDatos(f"Error de base de datos: {error}")
```

Todas las excepciones heredan de `EcoTechError`, de modo que el programa puede atrapar «cualquier error del sistema» con una sola clase o una específica cuando importa. `DatoInvalidoError` también hereda de `ValueError`, por lo que es compatible con el manejo estándar de Python. El repositorio captura las excepciones crudas de sqlite3 (por ejemplo `sqlite3.IntegrityError` ante un id o correo repetido) y las **relanza traducidas** al vocabulario del negocio (`RegistroDuplicadoError`, `ErrorBaseDatos`) con `raise ... from error`. Así el resto del programa no depende de sqlite3.

5.2 Bloques try - except Específicos y Validación de Entradas

```
# main.py: validación de tipo en la entrada
def pedir_entero(mensaje):
    while True:
        try:
            return int(input(mensaje))
        except ValueError:
            print("  Ingresa un número entero válido.")

# main.py: el menú atrapa los errores y sigue funcionando
    try:
        opciones[eleccion][1]()
    except EcoTechError as error:
        print(f"No se pudo completar la operación: {error}")
    except ValueError as error:
        print(f"Dato inválido: {error}")
```

Cada bloque `try-except` captura una falla concreta: `ValueError` cuando el usuario escribe texto donde va un número o una fecha mal formada (se avisa y se vuelve a pedir el dato); `sqlite3.Error` en la conexión y en las consultas (se traduce a `ErrorConexionBD` o `ErrorBaseDatos`); `sqlite3.IntegrityError` ante un registro duplicado (`RegistroDuplicadoError`); `EcoTechError` en el menú (se muestra el mensaje y se vuelve al menú); `KeyboardInterrupt` y `EOFError` para cerrar la sesión ordenadamente; y `ValueError` en `GestorCifrado.validar()` cuando el hash guardado tiene un formato inválido (devuelve `False`).

Por qué excepciones específicas: cada falla requiere una respuesta distinta (volver a pedir un dato, avisar que el id ya existe, deshacer una transacción). Un `except:` desnudo o un `except Exception: pass` esconde los errores, impide depurar y deja el sistema en un estado desconocido, por eso están prohibidos y no existen en el proyecto (verificado con una búsqueda de texto en todo el código).

Validación de entradas en tres niveles: en `main.py` se valida el *tipo* (entero, decimal, fecha); en el dominio, la *regla de negocio* (sueldo mayor a 0, correo con usuario y dominio, clave de al menos 8 caracteres, jornada entre 0 y 24 horas); y en la base, las restricciones `CHECK` y `UNIQUE`. Resultado real de las pruebas de validación (paso 3 de la demostración; el programa no se interrumpe):

```
>>> 3. Validaciones y manejo de errores (el programa no se interrumpe)
  Sueldo negativo: DatoInvalidoError -> El sueldo base debe ser un número mayor a 0.
  Correo inválido: DatoInvalidoError -> El correo no tiene un formato válido (ej:
nombre@dominio.cl).
  Id repetido: RegistroDuplicadoError -> Ya existe un registro con esos datos únicos.
  Empleado inexistente: RegistroNoEncontradoError -> No existe un empleado activo con id 999.
```

Docente: Noslin A. Guerra Cabeza
INACAP, Sede San Pedro de la Paz

5.3 Operación Transaccional Atómica (Commit y Rollback)

Momento Transaccional	Línea de Código / Operación	Efecto en la Base de Datos
1. Inicio de transacción	`try:` (SQLite abre la transacción en la primera escritura)	Agrupa las operaciones: aún no son permanentes.
2. Primera escritura	`conn.execute(self._SQL_INSERT, self._parametros(empleado))`	Inserta el registro del empleado (tabla padre) dentro de la transacción.
3. Segunda escritura	`conn.execute("INSERT INTO credencial ... VALUES (?, ?, ?, ?, ?)", (...))`	Inserta la credencial dependiente del empleado.
4. Caso de éxito	`conn.commit()`	Ambas operaciones se consolidan en disco.
5. Caso de fallo	`except sqlite3.Error: conn.rollback()`	Se deshace la primera escritura; la base queda intacta.

Operación atómica: `registrar_con_credencial(empleado, credencial)`. Registrar un empleado y su credencial son dos escrituras que deben ocurrir juntas o no ocurrir: no puede existir una credencial sin empleado, ni un empleado a medio registrar.

```
# repositorio/empleado_repo.py
    def registrar_con_credencial(self, empleado, credencial):
        """Operación atómica: o se guardan el empleado Y su credencial, o no se guarda nada."""
        conn = self._bd.conexion
        try:
            conn.execute(self._SQL_INSERT, self._parametros(empleado))  # 1ª escritura
            conn.execute("INSERT INTO credencial (nombre_usuario, clave_cifrada, habilitada, perfil,
id_persona) "
                         "VALUES (?, ?, ?, ?, ?)",  # 2ª escritura
                         (credencial.nombreUsuario, credencial.claveCifrada,
                          int(credencial.habilitada), credencial.perfil.etiqueta,
empleado.idPersona))
            conn.commit()  # todo salió bien
        except sqlite3.Error as error:
            conn.rollback()  # deshace la 1ª escritura
            raise self.traducir_error(error) from error
```

Demostración experimental (`python main.py --demo`, paso 5): primero se registra un empleado con su credencial (éxito). Luego se intenta registrar otro empleado con un nombre de usuario que ya existe: la primera escritura se ejecuta, la segunda falla con `RegistroDuplicadoError`, se ejecuta `rollback()` y se comprueba que el empleado de la primera escritura **no quedó guardado**.

```
>>> 5. Transacción atómica (empleado + credencial)
Caso éxito: empleado 10 guardado -> Jesús Capetillo
Caso fallo: la 2ª escritura falló (Ya existe un registro con esos datos únicos.)
Rollback comprobado: el empleado 11 NO quedó guardado (la BD sigue consistente).
```

Docente: Noslin A. Guerra Cabeza
INACAP, Sede San Pedro de la Paz

6. Auditoría y Validación Crítica de Código Generado por IA

6.1 Registro de Herramientas e Interacciones con IA

Herramienta y Versión	Fecha de Consulta	Propósito de la Consulta	Resultado Preliminar
Claude (Anthropic), modelo Claude Sonnet, vía claude.ai	Octubre 2026 (avance formativo el 7 de octubre; ES2 el 20 de octubre)	Redactar y organizar el código a partir del UML validado: clases del dominio, capa de repositorio con consultas parametrizadas, excepciones, transacción y menú; además apoyo para explicar el código.	Entregó código funcional y estructurado, pero su primer borrador tenía debilidades de validación y seguridad que el equipo revisó y corrigió (ver 6.2).

> **Declaración de uso:** el código fue redactado con apoyo de Claude y luego revisado, ejecutado y probado por el equipo (demostración automática, recorrido del menú con datos inválidos, prueba de inyección SQL y fallo forzado para comprobar el rollback).

6.2 Matriz de Auditoría y Decisiones Técnicas

Mínimo exigido: 4 fragmentos auditados. Es obligatorio reportar al menos una modificación o descarte por motivos de seguridad.

Fragmento o Sugerencia de la IA	Aspecto Analizado	Decisión	Criterio Técnico y Corrección Aplicada
`Credencial` del primer borrador aceptaba cualquier clave, incluso muy corta o vacía	Seguridad	Modifico	Se exige un mínimo de 8 caracteres y se rechaza la clave vacía, con `DatoInvalidoError` (dominio/credencial.py). Una clave débil o vacía comprometería la autenticación.
`GestorCifrado.validar` fallaba con un error no controlado si el hash guardado tenía mal formato	Seguridad y estabilidad	Modifico	Se captura `ValueError` y se devuelve `False`; un dato corrupto en la base no tumba el programa ni revela información (dominio/gestor_cifrado.py).
`Persona` aceptaba cualquier id y un correo válido solo con «@»	Consistencia y validación	Modifico	Id entero mayor a 0 y correo con usuario y dominio; las validaciones se centralizaron en `validaciones.py` para reutilizarlas (dominio/validaciones.py).
Clase de conexión que ejecutaba SQL y convertía todo error de sqlite3 en un `RuntimeError` genérico	Modularidad y manejo de errores	Modifico	El SQL pasó a repositorios; `BaseRepositorio` ejecuta con parámetros, hace rollback y traduce los errores a excepciones del negocio (repositorio/base_repo.py).
Idea de usar `python-dotenv` para leer el archivo `.env`	Dependencias y seguridad	Descarto	Agrega una librería externa para una tarea simple; se escribió un lector propio de pocas líneas (`cargar_env`) y el proyecto queda solo con la biblioteca estándar.
`ReportePDF` y `ReporteExcel` generan un PDF mínimo y texto tabulado	Alcance y coherencia con UML	Acepto con límite	Cumplen el contrato `exportar()` del UML; un PDF o .xlsx real requiere una librería externa y queda como mejora futura, declarada como tal.

7. Trazabilidad de Aportes y Preparación de la Defensa Individual

7.1 Matriz de Contribución y Autoría del Código

Estudiante (Nombre y RUT)	Módulos / Archivos Desarrollados	% Contribución Estimada	Firma de Compromiso
Amanda Hidalgo (líder) 21694333-3	Desarrollo compartido de todos los módulos (dominio, repositorio y menú), en partes iguales	33,3 %	
Roberto Gajardo 21582058-0	Desarrollo compartido de todos los módulos (dominio, repositorio y menú), en partes iguales	33,3 %	
Jesús Capetillo 22711930-6	Desarrollo compartido de todos los módulos (dominio, repositorio y menú), en partes iguales	33,3 %	

7.2 Registro de Gestión del Líder de Proyecto

> La líder del proyecto, Amanda Hidalgo, coordinó el trabajo del equipo. Las tareas se repartieron de forma equitativa: los tres integrantes participaron en todas las etapas del proyecto (revisión del UML de Roberto, programación de las clases, repositorios y menú, pruebas y preparación de la defensa), sin dividir el trabajo por módulos, de modo que cada uno conoce y puede explicar todo el código.

7.3 Cuestionario de Preparación para la Defensa Individual

N°	Pregunta Clave de la Defensa	Aspecto Evaluado	Concepto Clave que se Debe Explicar
1	«Muéstrame una consulta parametrizada tuya y explícame qué pasaría si usaras concatenación con `' OR '1'='1`»	Seguridad de Datos (2.1.3)	Aislamiento de código vs datos en el motor relacional.
2	«¿Dónde está implementada la herencia y por qué se usó `super()` en esa clase?»	POO y Diseño (2.1.2)	Reutilización de inicializador base y consistencia con UML.
3	«Si esta operación falla a la mitad, ¿qué evita que la base de datos quede corrupta?»	Transacciones (2.1.4)	Atomicidad transaccional: ejecución de rollback ante excepción.
4	«¿Qué fragmento de código te propuso la IA que tuviste que corregir o descartar?»	Validación de IA (2.1.5)	Criterio técnico: seguridad (SQL), manejo de errores o arquitectura.
5	«Si el cliente pide cambiar de SQLite a PostgreSQL, ¿cuántos y cuáles archivos debes modificar?»	Arquitectura / Repositorio	Independencia de capas: solo se modifica el módulo repositorio; el dominio queda intacto.

8. Referencias Bibliográficas

- Anthropic. (2026). Claude [Modelo de lenguaje de gran tamaño]. https://claude.ai
- Python Software Foundation. (2026). hashlib — Secure hashes and message digests. Python Official Documentation.
- Python Software Foundation. (2026). *sqlite3 — DB-API 2.0 interface for SQLite databases*. Python Official Documentation.